

Journal Pre-proofs

Full Length Article

Gradient-based trajectory planner of real-time obstacle avoidance for fixed-wing unmanned aerial vehicles in cluttered environments

Ziyu DU, Guangtong XU, Zhu WANG, Ziyang MENG

PII: S1000-9361(26)00045-2
DOI: <https://doi.org/10.1016/j.cja.2026.104108>
Reference: CJA 104108

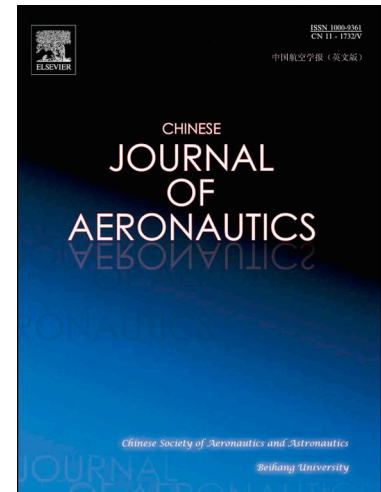
To appear in: *Chinese Journal of Aeronautics*

Received Date: 26 May 2025
Revised Date: 30 January 2026
Accepted Date: 31 January 2026

Please cite this article as: Z. DU, G. XU, Z. WANG, Z. MENG, Gradient-based trajectory planner of real-time obstacle avoidance for fixed-wing unmanned aerial vehicles in cluttered environments, *Chinese Journal of Aeronautics* (2026), doi: <https://doi.org/10.1016/j.cja.2026.104108>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2026 The Author(s). Published by Elsevier Ltd on behalf of Chinese Society of Aeronautics and Astronautics.



Contents lists available at [ScienceDirect](https://www.sciencedirect.com)

Chinese Journal of Aeronautics

Journal homepage: www.elsevier.com/locate/cja

FULL LENGTH ARTICLE

Gradient-based trajectory planner of real-time obstacle avoidance for fixed-wing unmanned aerial vehicles in cluttered environments

Ziyu DU^a, Guangtong XU^{b,*}, Zhu WANG^c, Ziyang MENG^{a,*}^aDepartment of Precision Instrument, Tsinghua University, Beijing 100084, China^bHuzhou Institute, Zhejiang University, Huzhou 313000, China^cDepartment of Automation, North China Electric Power University (Baoding), Baoding 071003, China

Received 26 May 2025; revised 11 August 2025; accepted 23 November 2025

Abstract

Autonomous flight of fixed-wing Unmanned Aerial Vehicles (UAVs) in cluttered environments demands a trajectory planner that offers low computational overhead while ensuring high robustness. However, trajectory planning problems subject to nonlinear dynamics and obstacle avoidance constraints, especially with irregularly shaped obstacles, often impose prohibitive computational costs on existing algorithms. To achieve efficient and robust trajectory generation for fixed-wing UAVs, this paper proposes a multi-stage trajectory planning method leveraging B-spline curve representation and gradient-based optimization. Initially, the A* algorithm is modified to search for collision-free control points on an occupancy grid map, taking into account various irregular obstacles. Subsequently, the B-spline-based trajectory optimization problem is formulated, incorporating obstacle avoidance constraints via the Euclidean signed distance field. A minimum-curvature-limited trajectory refinement stage is designed with a curvature adjustment mechanism to ensure dynamic feasibility. Simulation results demonstrate that this planner achieves over an order of magnitude speedup compared to state-of-the-art optimization-based approaches in global planning scenarios. Furthermore, the method is implemented into a local replanning framework to validate its real-time obstacle avoidance ability, where re-planning consumes only tens of milliseconds even in cluttered environments. Trajectory tracking experiments in simulation and outdoor flight experiments with a physical fixed-wing UAV (Sky Surfer X8P) both verify the executability of the generated trajectories by the proposed planner.

Keywords: Fixed wings; Real-time trajectory planning; Gradient descent optimization; Differential flatness; Dynamical feasibility

*Corresponding author. E-mail address: guangtong_xu@163.com, ziyangmeng@tsinghua.edu.cn

1. Introduction

Miniature fixed-wing UAVs have been employed in a variety of applications, including search and rescue, express delivery, reconnaissance, and surveillance, due to their compact size, extended endurance, and low cost [1]. Efficient and robust trajectory planner becomes a critical component of ensuring safe autonomy for UAVs, as it involves identifying a safe temporal route between an initial and final states on constraint manifolds [2]. In practical scenarios at low altitudes, particularly in urban and mountainous regions, fixed-wing UAVs encounter cluttered environments

that present significant challenges to autonomous flight. Consequently, it is essential for UAVs to have real-time obstacle avoidance capabilities. Although there exist numerous trajectory planning methods, the real-time solution for dynamically-feasible and safe trajectories in cluttered environments remains a technological bottleneck.

Trajectory planning methods for fixed-wing UAVs can be broadly classified into four categories: geometric route search-based, random sampling-based, potential field-based, optimization-based, and learning-based methods. These approaches have been extensively researched and mutually combined to produce efficient planning strategies. Geometric route search and sampling-based methods [3] offer completeness and high efficiency, but generally do not account for dynamic feasibility. In contrast, optimization-based methods [4] have been shown to satisfy nonlinear dynamic constraints. However, optimization methods are sensitive to initial guesses, and the computational cost of nonlinear optimization is high, making them less suitable for real-time applications. For instance, nonlinear dynamic constraints and high-dimensional state space are incorporated into the optimization framework, using a fourth-order Runge-Kutta method for solution [5]. This method requires over 1 hour of computation to generate a 24.11 seconds flight trajectory. The Pseudospectral method [6] can solve the trajectory planning problem, but typical solvers like GPOPS-II [7] still consume significant computational resources (36.29 seconds) to plan a safe trajectory in a simple environment with only 6 cylindrical obstacles. Learning-based methods have been investigated to enhance the planning performance from environmental data or interactions. Chai et al. [8] aims to enable real-time reentry trajectory planning of hypersonic vehicles, they design a deep neural network to generate optimal commands online. Wu et al. [9] propose a deep-reinforcement-learning-based reactive online obstacle avoidance method to provide high trackability and efficiency. Yan et al. [10] introduce a multiagent deep reinforcement learning approach to learn the decentralized flocking with obstacle avoidance policy for multiple fixed-wing UAVs. The advantage of learning-based approach lies in its ability to optimize high-dimensional control strategies through data as well as its capability to handle strongly nonlinear dynamic systems. Besides, the trained network can offer real-time inference preformation for online applications. Nevertheless, learning-based method requires a large amount of high-quality data, which is difficult to collect. Existing learning-based methods perform well in scenarios covered by the data, but their generalization ability in new environments is limited. Besides, neural network suffers from a lack of interpretability, and the failures of planning may occur occasionally.

In this paper, the optimization-based method is introduced due to its deterministic characteristic and well-studied optimization solver. To alleviate the computationally expensive issue of optimization-based methods, Sequential Convex Programming (SCP) [11, 12] emerges as an attractive option for efficient trajectory generation. SCP converts nonlinear optimization into a series of convex programming subproblems, yet the convexification has approximate error and may produce infeasible solutions. Wang et al. [13] applied SCP on UAV trajectory planning, and the computation time decreased to 9.61s. Bonalli et al. [14] proposed the Guaranteed Sequential Trajectory Optimization (GuSTO) to reduce the risk of trapping into local minimum and improve convergence. Xu et al. [15] developed Trust-Region Filtered SCP (TRF-SCP) to construct low-complicated convex programming subproblems. Even though TRF-SCP has obvious efficiency advantage compared with SCP and GuSTO, it still costs seconds and is not suited for the deployment on dense obstacle environments. Particularly for cluttered environments, the SCP-based trajectory iteration progresses towards solution conservatively and even cannot converge to at least a feasible trajectory.

Aiming at addressing the issue of low solution efficiency or even infeasibility due to the sensitivity to initial guesses of optimization-based methods, researches have tended to adopt a hierarchical planning strategy [16]. For these approaches, geometric search methods are first used to generate feasible obstacle-avoidance path, and then optimization methods are employed to handle multi-objective optimization problems, such as safety and dynamic feasibility. Zhou et al. [17] developed robust and efficient trajectory generation method integrating hybrid-A*-based kinodynamic path search and B-spline optimization for flight in unknown environments. Liu et al. [18] proposed a search-based trajectory planning method, and the optimal trajectory was generated by deriving the explicit solution of a linear quadratic minimum time problem. Ding et al. [19] investigated a hierarchical planning method incorporating B-spline-based kinodynamic searching and elastic optimization. Tordesillas et al. [20] presented the fast and safe trajectory planner using front-end jump point search and back-end optimization. These aforementioned planning methods have remarkable computation efficiency and robustness advantages. However, these methods are designed for quadrotors specifically and cannot be applied on fixed-wing UAVs conveniently. Li et al. [21] designed a flat mapping of the fixed-wing UAV's overload and flight path angle to the planning space of the Minimum Control (MINCO) trajectory [22], enabling efficient trajectory solving within the parameterized space of polynomial trajectories. Essentially, this work still focuses on constraints of velocity and force, lacking consideration for attitude control. It plans trajectories in a known environment without a receding-horizon scheme planner, leaving a gap to real-time obstacle avoidance.

The real-time performance of fixed-wing UAV trajectory planner depends on assumptions about the UAV's dynamics constraints and environmental perception. Overly restrictive flight motion assumptions can result in conservative trajectories, while high-dimensional state spaces or complex dynamics can degrade the real-time efficiency of optimization problems. Chitsaz and LaValle [23] employ Dubins curves and derive time-optimal

trajectories based on Pontryagin's Maximum Principle. The fixed-wing UAV trajectories are constrained to turns with a minimum radius, straight line segments, and pieces of planar elastica. Such trajectory planning methods face significant challenges in irregular and high-density environments (e.g., low-altitude urban settings), where strict safety and flexibility requirements exist. Another approach of achieving autonomous flight for fixed-wing UAVs seeks to decompose motion into motion primitives and plan the trajectory using a high-level guidance law. This method effectively incorporates the UAV's dynamics and enables fast flight in cluttered environments [24], but it faces trade-offs between search space dimensions and optimality of the generated trajectories. Using a full-dimensional state space in Model Predictive Control (MPC) methods leads to a different extreme [25], where the nonlinear dynamics and high-dimensional state space prevent the algorithm from real-time requirements. The introduction of differential flatness provides a clue for decoupling and reducing the dimension of optimization problems. It allows the planning problem to be transformed into an optimization over flat outputs in a linear space, without the need to solve for the entire states of the original dynamics. Planners incorporating differential flatness have demonstrated promising results on platforms like quadrotors [26] and car-like robots [27]. Fixed-wing UAV planners using differentially flat trajectory representations can achieve safe flight while ensuring real-time performance and satisfaction of dynamic constraints.

In addition to computation efficiency, fixed-wing UAV planners, when applied to real-time obstacle avoidance in cluttered environments, face another key issue: their strong assumptions about environmental perception. Most of previous methods represent obstacles with simple shapes, such as manually refined cylinders or convex polygons. GPOPS-II [7] assumes cylindrical obstacles, while GuSTO [14] models obstacles as convex polyhedra. Modeling obstacles with regular shapes allows optimization algorithms to define safe feasible regions for solution space, which improves solution efficiency and guarantees local optimality. Bry et al. [28] proposed a method that does not rely on convex feasible regions defined by regular obstacles. However, this method does not offer a receding horizon module for planning trajectories in 3D unknown environments. It still depends on pre-constructed 2D maps, resulting that it cannot guide a fixed-wing UAV to perform real-time autonomous flight.

To address the trade-off between flight agility and the high-dimensional state space for real-time flight in cluttered environments, we introduce differential flatness of fixed-wing UAVs into the B-spline-based trajectory planning module. In order to achieve a receding horizon-style local planner, we divide the trajectory planning of fixed-wing UAVs into three stages: trajectory initialization, optimization, and refinement. Specifically for the proposed planner, the scale and positions of control points are introduced to formulate the optimization problem instead of discretizing states and controls directly. Moreover, the differential flatness of fixed-wing UAVs is exploited to ensure dynamical feasibility. This multi-stage planner can mitigate the sensitivity to initial guesses and prevent trapping into local minimum. Extensive experiments are performed to validate the planning performance. We summarize the main contributions as:

- (1) A systematic trajectory planner for fixed-wing UAVs integrating A*-driven trajectory initialization, B-spline-based trajectory optimization, and minimum-curvature-limited trajectory refinement is proposed to enhance the computation efficiency and robustness.
- (2) The trajectory optimization problem is formulated considering smoothness, state and control bound, and obstacle avoidance. To guarantee dynamical feasibility, the time reallocation mechanism and curvature adjustment strategy are designed according to the differential flatness.

In what follows, an overview of the multi-stage trajectory planner is illustrated in Section 2. Sections 3, 4, and 5 present the trajectory initialization, optimization, and refinement, respectively. Numerical simulations on global planning and local planning frameworks are provided in Sections 6 and 7, respectively. Outdoor flight experiments of trajectory tracking are conducted in Section 8. Section 9 concludes the strengths of this method and provides the future work.

2. Overview

The framework of the systematic trajectory planning method for fixed-wing UAVs is illustrated in Fig. 1. We design a three-stage trajectory planning method integrating trajectory initialization, trajectory optimization, and trajectory refinement. The details are described as follows.

Trajectory Initialization: In the 1st stage, A* is adopted to generate a collision-free route on the occupancy gridmap. Then, a series of discretized route points with a constant interval are selected as the control points for B-spline to conduct the following optimization problem. This stage aims to offer well-defined initial guesses, and prevent the optimization from trapping into local minimum.

Trajectory Optimization: In the 2nd stage, the B-spline-based trajectory optimization problem subject to initial/final states, velocity bounds, and obstacle avoidance is formulated. The control points of B-spline are regarded as design variables. ESDF map is used to consider the obstacle-avoidance constraint. This stage leverages the continuity, smoothness, and conciseness merits of B-spline to improve the planning robustness.

Trajectory Refinement: Based on the generated B-spline, the complete state and control variables in the UAV

dynamics are derived via differential flatness in Eq.(2). The bound conditions on states and controls are checked, and the time reallocation is conducted to provide reasonable time scale. Then, the B-spline with updated time is re-optimized subject to minimum curvature limitation induced by the dynamical constraint. Although the 2nd stage ignores the dynamics in Eq.(1), this stage can guarantee the dynamical feasibility, and saves overall planning runtime considerably.

The proposed method addresses the trajectory planning problem by separately optimizing in the spatial and temporal domains. The spatial optimization occurs first in the trajectory optimization stage, where B-spline control points are adjusted for smoothness and collision avoidance. Then, the curvature adjustment step further modifies the control point positions to meet curvature constraints. Temporal optimization is conducted exclusively during the trajectory refinement stage through a time reallocation mechanism, which scales the trajectory's duration to ensure that control inputs, such as thrust and lift factors, remain feasible. This decoupling approach is motivated by the inherent complexity of jointly optimizing in the full spatiotemporal state space, particularly under complex nonlinear dynamic constraints. Such joint optimization often results in prohibitive computational costs, making it unsuitable for real-time applications.

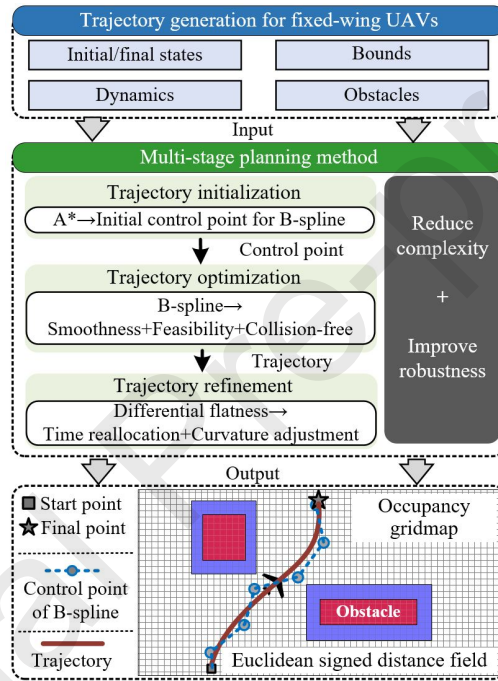


Fig. 1 Framework of multi-stage trajectory planning method.

In this paper, the dynamics of fixed-wing UAVs [29] with a zero-sideslip angle are expressed in the Cartesian coordinate system, as shown in Eq.(1). The states $s(t)$ include 3D position $p = [p_x(t), p_y(t), p_z(t)]^T$, flight velocity $v(t)$, heading angle $\vartheta(t)$, and flight-path angle $\gamma(t)$. The controls $c(t)$ involve thrust $T(t)$, bank angle $\phi(t)$, load factor $n(t) = L(t)/(mg)$, and $L(t)$ denotes the total lift.

$$\begin{cases} \dot{p}_x(t) = v(t)\cos\gamma(t)\cos\vartheta(t) \\ \dot{p}_y(t) = v(t)\cos\gamma(t)\sin\vartheta(t) \\ \dot{p}_z(t) = v(t)\sin\gamma(t) \\ \dot{v}(t) = (T(t) - D(t)) / m - g\sin\gamma(t) \\ \dot{\vartheta}(t) = (g / v(t)) (n(t)\sin\phi(t) / \cos\gamma(t)) \\ \dot{\gamma}(t) = (g / v(t)) (n(t)\cos\phi(t) - \cos\gamma(t)) \end{cases} \quad (1)$$

where m , t , g , and $D(t) = 1/2\rho S C_D v(t)^2$ denote the mass, flight time, acceleration of gravity, and aerodynamic drag. ρ , S , and $C_D = C_{D0} + k_0 C_L^2$ are the air density, wing area, and drag coefficient. C_{D0} , $k_0 = 1/(\pi e A_R)$, and

$C_L = 2n(t)mg / (\rho S v(t)^2)$ are the drag coefficient at zero lift, lift-induced drag coefficient, and lift coefficient. A_R and e denote the aspect ratio and Oswald number, respectively.

For the dynamics of Eq.(1), the states and controls can be written as algebraic functions of three carefully selected flat outputs $\mathbf{p} = [p_x(t), p_y(t), p_z(t)]^T$ and corresponding derivatives shown in Eq.(2), which is defined as the differential flatness of fixed-wing UAV dynamics [30].

$$\begin{cases} v(t) = (\dot{p}_x(t)^2 + \dot{p}_y(t)^2 + \dot{p}_z(t)^2)^{\frac{1}{2}} \\ \gamma(t) = \arcsin(\dot{p}_z(t) / v(t)) \\ \vartheta(t) = \arcsin(\dot{p}_y(t) / (v(t)\cos\gamma(t))) \\ T(t) = D(t) + m\dot{v}(t) + mg\sin\gamma(t) \\ \phi(t) = \arctan \frac{\dot{p}_x(t)v(t)\cos\gamma(t)}{g\cos\gamma(t) + v(t)\dot{\gamma}(t)} \\ n(t) = (g\cos\gamma(t) + v(t)\dot{\gamma}(t)) / g\cos\phi(t) \end{cases} \quad (2)$$

3. Trajectory initialization

B-spline [30] is employed to represent the flight trajectory, and A* is introduced to search collision-free route. The control points are extracted from the route as the initial guesses. Consider a B-spline curve of degree p with $K+1$ control points $\mathbf{Q} = [\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_K]$ and knot vector $\mathbf{u} = [u_0, u_1, \dots, u_M]$, where $M = K+p+1$, the corresponding trajectory $\mathbf{B}(\tau)$ is expressed as

$$\mathbf{B}(\tau) = \sum_{i=0}^K \mu_{i,p}(\tau) \mathbf{Q}_i \quad (3)$$

where τ and $\mathbf{Q}_i = [p_{i,x}, p_{i,y}, p_{i,z}]^T$ denote the trajectory time and 3D position of control points. $\mu_{i,p}(\tau)$ is the basis function, and we refer readers to Ref. 31 for more details.

In a clamped B-spline, the first and last control points are repeated for $p+1$ times. Thus, it can inherently guarantee that the represented trajectory satisfies the initial and final position constraints. B-spline has the advantages of convex hull property, local modification feature, and continuity. In addition, the derivative of B-spline still belongs to B-spline. Note that we use the uniform B-spline, i.e., the interval between knots $(u_{i+1} - u_i)$ is a constant.

To generate the initial control points efficiently, the standard expanding pattern of A* is modified to search a collision-free route on a occupancy grid map, as shown in Fig. 2. Considering the turning radius restriction of fixed-wing UAVs, the expanding directions exceeding maximum turning angle are eliminated. We sample $K+1$ sparse discretized points with constant step size Δp along the route as the initial guesses for the following trajectory optimization.

A larger grid inherently increases the search space of the A* algorithm. Assume the 3D map grid has dimensions of $L \times W \times H$. The total number of cells (or voxels) in the map is $N_g = L \times W \times H$. The overall time complexity of the A* algorithm is approximately $O(N_g \cdot \lg N_g)$. This means that the computation time of the A* algorithm has a superlinear relationship with the total volume N of the map grid. A higher resolution (i.e., smaller grid cells) provides more detailed environmental information, enabling the algorithm to plan paths more accurately and thus increasing the success rate. However, this increased resolution also increases computational complexity, resulting in longer search time. Therefore, we select a resolution of 1 m for a 200 m $\times$ 200 m $\times$ 50 m map to balance the computation accuracy and efficiency.

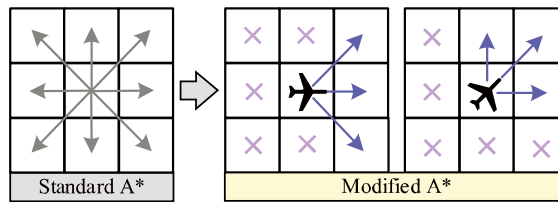


Fig. 2 Expanding process of modified A*.

4. Trajectory optimization

In this section, the B-spline-based trajectory generation is formulated as an unconstrained optimization problem, and the initial B-spline is optimized to obtain smooth trajectory obeying obstacle-avoidance constraints and bound conditions on velocity and acceleration.

The core of the formulation involves a penalty method where all constraints are incorporated as soft constraints within a composite objective function. This approach avoids the computational overhead associated with complex solvers and is better suited for real-time replanning. Design variables to be optimized are positions of control points.

Design variables are control points $\{\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_K\}$. The objective function is designed in Eq.(4) to trade off the smoothness cost J_s , bound cost J_b , and collision cost J_c .

$$\min_{\mathbf{Q}} J = \lambda_s J_s + \lambda_b J_b + \lambda_c J_c \quad (4)$$

where λ_s , λ_b , and λ_c indicate the coefficients of smoothness cost, bound cost, and collision cost, respectively. The smoothness cost J_s penalizes the integral of squared acceleration and jerk, enforcing trajectory smoothness which is critical for executability. The bound cost J_b penalize violations of predefined velocity and acceleration bounds. The collision cost J_c handles environmental constraints by leveraging a precomputed ESDF map to penalize proximity to obstacles, ensuring the trajectory remains safe. The typical gradient descent optimization algorithm can solve the unconstrained optimization problem efficiently.

4.1. Smoothness cost

The first, second, and third order derivatives of B-spline become the velocity, acceleration, and jerk of the trajectory. Since the derivative of a B-spline is still a B-spline, the velocity, acceleration, and jerk can be uniquely determined by their corresponding control points $\mathbf{V}_i$, $\mathbf{A}_i$, and $\mathbf{J}_i$, obtained from $\mathbf{Q}_i$ and Eq.(5).

$$\mathbf{V}_i = (\mathbf{Q}_{i+1} - \mathbf{Q}_i)\beta, \mathbf{A}_i = (\mathbf{V}_{i+1} - \mathbf{V}_i)\beta, \mathbf{J}_i = (\mathbf{A}_{i+1} - \mathbf{A}_i)\beta \quad (5)$$

where time scale $\beta = \tau / t$ is defined as the ratio of τ and practical flight time t . The smoothness cost aims to minimize the acceleration and jerk of the trajectory, and is given as

$$J_s = \sum_{i=0}^{K-2} \|\mathbf{A}_i\|^2 + \sum_{i=0}^{K-3} \|\mathbf{J}_i\|^2 \quad (6)$$

where $\|\cdot\|$ denotes the Euclidean norm.

4.2. Bound cost

According to the dynamics property of fixed-wing UAVs, a lower bound constraint on flight velocity is set to prevent UAVs from stalling. The minimum velocity limitation $v_{\min}$ is obtained by using Eq.(7).

$$v_{\min} = (2mg / \rho S)^{\frac{1}{2}} (k_0 / C_{D0})^{\frac{1}{4}} \quad (7)$$

Furthermore, there exist maximum velocity $v_{\max}$ and acceleration $a_{\max}$ restrictions in view of aerodynamic drag and maximum thrust of propellers. The bound cost on velocity and acceleration is designed as Eq.(8).

$$\mathbf{M}_v(\mathbf{V}_i) = \begin{cases} (v_{\min} - \|\mathbf{V}_i\|)^2, & \|\mathbf{V}_i\| \leq v_{\min} \\ 0, & v_{\min} < \|\mathbf{V}_i\| < v_{\max} \\ (\|\mathbf{V}_i\| - v_{\max})^2, & \|\mathbf{V}_i\| \geq v_{\max} \end{cases} \quad (8)$$

$$\mathbf{M}_a(\mathbf{A}_i) = \begin{cases} 0, & \|\mathbf{A}_i\| < a_{\max} \\ (\|\mathbf{A}_i\| - a_{\max})^2, & \|\mathbf{A}_i\| \geq a_{\max} \end{cases}$$

The total bound cost is shown in Eq.(9), where w_v and w_a are penalty coefficients.

$$J_b = \sum_{i=0}^{K-1} w_v \mathbf{M}_v(\mathbf{V}_i) + \sum_{i=0}^{K-2} w_a \mathbf{M}_a(\mathbf{A}_i) \quad (9)$$

4.3. Collision cost

The optimized trajectory must be collision-free in obstacle environments to guarantee the safety of UAVs. The collision cost J_c based on ESDF map is designed. This map stores the distance to obstacles or free space, and is used to calculate the gradient information for collision avoidance. ESDF map is expressed as Eq.(10) for each control point $\mathbf{Q}_i$.

$$\text{ESDF}(\mathbf{Q}_i) = \begin{cases} \|\mathbf{Q}_i - \mathbf{p}_o\|, & \text{if } \mathbf{Q}_i \text{ is unoccupied} \\ -\|\mathbf{Q}_i - \mathbf{p}_u\|, & \text{if } \mathbf{Q}_i \text{ is occupied} \end{cases} \quad (10)$$

where $\mathbf{p}_o$ denotes the nearest occupied voxel with respect to $\mathbf{Q}_i$, meanwhile $\mathbf{p}_u$ is the nearest unoccupied voxel. The collision cost of control point $\mathbf{Q}_i$ is defined as a piecewise function in Eq.(11).

$$d_{c,i} = r_s - \text{ESDF}(\mathbf{Q}_i)$$

$$j_c(d_{c,i}) = \begin{cases} 0, & d_{c,i} \leq 0 \\ d_{c,i}^3, & 0 < d_{c,i} \leq r_s \\ 3r_s d_{c,i}(d_{c,i} - r_s) + r_s^3, & d_{c,i} > r_s \end{cases} \quad (11)$$

where r_s is the safety radius of fixed-wing UAVs and $d_{c,i}$ denotes the distance between the control point $\mathbf{Q}_i$ and free space. Because j_c belongs to a twice continuously differentiable function, its gradient becomes a quadratic function, which has appealing numerical stability. The total collision cost J_c of B-spline trajectory is the cumulative cost with respect to all the control points, as formulated in Eq. (12).

$$J_c = \sum_{i=0}^K j_c(d_{c,i}) \quad (12)$$

The computational complexity of generating the ESDF is $O(N_g)$, indicating that the time required for ESDF generation has a linear relationship with the total volume of the map. Consequently, if each dimension of the map is doubled, the ESDF generation time will increase by approximately eight times. To balance computational accuracy and cost, we maintain the same resolution as described in Section 3.

5. Trajectory refinement

In this stage, we design the time reallocation mechanism to make sure that the thrust $T(t)$ stays within a reasonable range. The curvature adjustment is proposed to limit the bank angle $\phi(t)$ by utilizing gradient descent optimization.

5.1. Time reallocation

The pseudocodes of time reallocation are listed in Algorithm 1. During the time reallocation process, we leverage differential flatness to calculate the thrust based on the trajectory through a closed-form solution. This thrust is subsequently utilized to facilitate the time reallocation. Due to that the closed-form solution costs slight runtime, it ensures the high efficiency in time reallocation.

Given the optimized B-spline trajectory, the thrust $T(\beta)$ and lift factor $n(\beta)$ can be deduced from the differential flatness in Eq. (2). To acquire reasonable time scale β , we define the proportion of exceeded control limits, formulated as Eq. (13).

$$\sigma(\beta) = \max\{T/T_{\max}, T_{\min}/T, n/n_{\max}, 1\} \quad (13)$$

Since $\sigma(\beta)$ is a non-monotonic piecewise function, we sample a series of β in the uniform sequence buffer_β with a fixed step $\Delta\beta$ from both sides of the initial time scale β_0 (lines 2-9). Then, we search for $\sigma(\beta)=1$, and reallocate β as the time scale for B-spline trajectory. If there exists no β making $\sigma(\beta)=1$, we choose β to minimize $\sigma(\beta)$, i.e., minimize the excess of control limits (lines 10-17).

Algorithm 1 is designed to ensure a low and predictable time consumption. The algorithm consists of two main loops: a sampling loop and an evaluation loop. In sampling loop (lines 2-9), the number of iterations is determined by the search range $\beta_{\max} - \beta_{\min}$ and the step size $\Delta\beta$. We define the number of samples as $N_s = \lceil (\beta_{\max} - \beta_{\min}) / \Delta\beta \rceil$. The complexity of this loop is $O(N_s)$. In evaluation Loop (lines 10-17), for each sample, it computes the thrust T and load factor n using the differential flatness in Eq.(2) through $K+1$ control points. The time consumption for each

sample is proportional to the control point number, i.e., the complexity is $O(K)$. The total complexity is therefore approximately $O(N_s \cdot K)$. Since N is a fixed parameters and K is constant for a given trajectory, the algorithm's runtime is bounded and deterministic. As for the robustness of Algorithm 1, it is inherently stable because it is not a recursive or iterative optimization that may fail to converge. It is a deterministic search that evaluates a predefined set of candidate time scales and selects the one that minimizes the violation of control limits.

Algorithm 1. Time reallocation.

Input: Control points $\mathbf{Q}$
lower and upper bound of search interval
 $\beta_{\min}$ and $\beta_{\max}$,
step size $\Delta\beta$, initial time scale β_0

Output: Reasonable time scale of B-spline $\hat{\beta}$

Parameters: Stack of search interval buffer_β

```

1.  $\text{buffer}_\beta \leftarrow \emptyset$ ;
2. while  $\beta_{\min} \leq \beta_0$  or  $\beta_{\max} \geq \beta_0$  do (Sampling loop)
3.   if  $\beta_{\min} \leq \beta_0$  then
4.      $\text{buffer}_\beta.\text{push}(\beta_{\min}); \beta_{\min} \leftarrow \beta_{\min} + \Delta\beta$ ;
5.   end if
6.   if  $\beta_{\max} \geq \beta_0$  then
7.      $\text{buffer}_\beta.\text{push}(\beta_{\max}); \beta_{\max} \leftarrow \beta_{\max} - \Delta\beta$ ;
8.   end if
9. end while
10. while  $\text{buffer}_\beta \neq \emptyset$  do (Evaluation loop)
11.    $\beta = \text{buffer}_\beta.\text{pop}()$ ;
12.    $T(\beta), n(\beta) \leftarrow \text{DifferentialFlatness}(\mathbf{Q}, \beta)$ ;
13.    $\sigma(\beta) \leftarrow \text{Eq. (13)}$ ;
14.   if  $\hat{\sigma} \geq \sigma(\beta)$  then
15.      $\hat{\sigma} \leftarrow \sigma(\beta); \hat{\beta} \leftarrow \beta$ ;
16.   end if
17. end while

```

5.2. Curvature adjustment

The curvature of the trajectory is generally constrained to a certain range to avoid large-angle rolling maneuvers for fixed-wing aircrafts. Here we propose a derivation to calculate the upper limit of the curvature and use it to optimize the position of control points. According to the differential flatness in Eq.(2), the roll angle $\phi(t)$ can be expressed in Eq.(14) with velocity $\mathbf{V}(t)$ and curvature $\kappa(t)$ of the trajectory when the pitch angle $\gamma(t)$ is relatively small and changes gently. Therefore, we can adjust the curvature to satisfy the maximum roll angle constraint.

$$\begin{aligned}
 \phi(t) &= \arctan \frac{\dot{\gamma}(t)v(t)\cos\gamma(t)}{g\cos\gamma(t) + v(t)\dot{\gamma}(t)} = \arctan \left(\frac{1}{v(t)} \cdot \frac{v^2(t)}{g} \right) \\
 &= \arctan \left(\frac{d\gamma(t)}{dt} \cdot \frac{dt}{ds} \cdot \frac{v^2(t)}{g} \right) = \arctan \frac{\kappa(t)v^2(t)}{g}
 \end{aligned} \tag{14}$$

The assumption to small pitch angle is consistent with the planner's intended operational scope, which is the en-route phase of flight rather than the takeoff and landing sequences. To provide a more rigorous justification on how sensitive the roll angle calculation is to variations in pitch, a theoretical analysis is performed. Starting from Eq.(14), we can derive the sensitivity at $\gamma=0$ as

$$\frac{\partial \phi}{\partial \gamma} \Big|_{\gamma=0} = \frac{\phi(t) \mathcal{R}(t) v(t)^2}{g^2} \quad (15)$$

At a scene where the UAV performs aggressive flight with a large roll angle, yaw rate or extreme speed, ϕ can be more sensitive to γ . However, the objective of the planner's design is to generate safe and efficient obstacle avoidance trajectories, rather than to push the maneuverability limits of the fixed-wing UAV. Therefore, aggressive flight scenarios are intentionally avoided by the planner's cost functions and constraints. Specifically, the air speed and yaw rate are constrained by the bound cost and smoothness cost in Eq.(4), while the roll angle is constrained by the curvature refinement process. As the above states and controls are maintained within a moderate, non-aggressive regime by the planner's overall design, the small-pitch-angle assumption is justified.

For a uniform B-spline $\tau \in [u_i, u_{i+1})$ where $u_{i+1} - u_i = 1$, we only limit its curvature at u_i , $(u_i + u_{i+1})/2$, and u_{i+1} to save computing resources. In this paper, we use a 3-order uniform B-spline curve, h denotes the normalized time variable $h = \tau - u_i$. The local curve $\mathbf{c}(h)$ in $\tau \in [u_i, u_{i+1})$ can be obtained by Eq.(15).

$$\mathbf{c}(h) = c_1 \mathbf{Q}_{i-3} + c_2 \mathbf{Q}_{i-2} + c_3 \mathbf{Q}_{i-1} + c_4 \mathbf{Q}_i \quad (16)$$

where the coefficients are defined as

$$\begin{cases} c_1 = (1-h)^3 / 6, c_2 = (3h^3 - 6h^2 + 4) / 6 \\ c_3 = (-3h^3 + 3h^2 + 3h + 1) / 6, c_4 = h^3 / 6 \end{cases} \quad (17)$$

The curvature of a 3 order B-spline curve is given as

$$\kappa(h) = |\mathcal{C}(h) \times \mathcal{C}(h)| / |\mathcal{C}(h)|^3 \quad (18)$$

where $\times$ and $|\cdot|$ denote cross product and absolute value. From the expression in Eq.(17), the positions at $h=0$, $h=0.5$, and $h=1$ are affine transformation with constant coefficients. Therefore, the curvature of local B-spline on these three points can be calculated with linear operations.

To generate dynamically feasible and safe trajectory, we conduct the optimization on control points to trade off the minimum curvature limitation and fitness of the preliminarily optimized trajectory in the 2nd stage. The cost function to perform gradient descent optimization is designed in Eq.(18).

$$\min_{\mathbf{Q}} J = \lambda_r J_r + \lambda_f J_f \quad (19)$$

where J_r and J_f represent the minimum curvature cost and fitness cost with respect to the input trajectory, respectively. λ_r and λ_f are the coefficients.

5.2.1. Minimum curvature cost

This cost is given by the sum of excess of curvature limitation over all control points, as expressed in Eq.(19).

$$J_r = \sum_{i=3}^K j_{r,i} \quad (20)$$

where $j_{r,i}$ is obtained as the sum of the excess at start $e_i(0)$, middle $e_i(0.5)$, and final points $e_i(1)$ of the curve controlled by every 4 control points. Fig. 3 depicts a graphical interpretation of curvature adjustment process.

$$j_{r,i} = e_i(0) + e_i(0.5) + e_i(1) \quad (21)$$

$$e_i(h) = \begin{cases} \kappa(h) - \kappa_{\max} & \kappa(h) > \kappa_{\max} \\ 0 & \text{otherwise} \end{cases} \quad (22)$$

An example of curvature adjustment is illustrated in Fig. 3. A local B-spline curve in $[u_i, u_{i+1})$ is determined by control points $\mathbf{Q}_{k-p}$, $\mathbf{Q}_{k-p+1}$, $\mathbf{Q}_{k-1}$, and $\mathbf{Q}_k$. The curvatures κ at u_i , $(u_{i+1} + u_i)/2$, and u_{i+1} are limited by the cost function J_f . In this example, since $\kappa_{0.5}$ exceeds $\kappa_{\max}$, the trajectory is punished to adjust the position of control

points and obtain the suitable $\kappa'_{0.5}$.

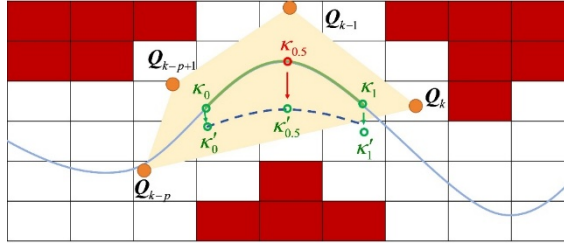


Fig. 3 Sketch of curvature adjustment process.

5.2.2. Fitness cost

This cost aims to punish the anisotropic displacements from the optimized trajectory in the 2nd stage to maintain the obstacle-avoidance configuration. Assume that $\mathbf{Q}_o = \{\mathbf{Q}_{o,0}, \mathbf{Q}_{o,1}, \dots, \mathbf{Q}_{o,K}\}$ are the optimized control points. The tangential and radical displacements ($d_{t,i}$, $d_{r,i}$) are given as

$$\begin{cases} d_{t,i} = (\mathbf{Q}_i - \mathbf{Q}_{o,i}) \cdot \frac{\mathbf{Q}_{o,i+1} - \mathbf{Q}_{o,i}}{\|\mathbf{Q}_{o,i+1} - \mathbf{Q}_{o,i}\|} \\ d_{r,i} = \left\| (\mathbf{Q}_i - \mathbf{Q}_{o,i}) \times \frac{\mathbf{Q}_{o,i+1} - \mathbf{Q}_{o,i}}{\|\mathbf{Q}_{o,i+1} - \mathbf{Q}_{o,i}\|} \right\| \end{cases} \quad (23)$$

The fitness cost J_f is formulated by the spheroidal metric with α and $\tilde{n}$ being the semi-major and semi-minor axis [32].

$$J_f = \sum_{i=0}^{K-1} (d_{t,i}^2 / \alpha^2 + d_{r,i}^2 / \tilde{n}^2) \quad (24)$$

The sketch of the fitness cost is shown in Fig. 4. Deviation along different direction results in anisotropic cost. A radical displacement of the UAV's trajectory is more likely to cause it to move closer to or collide with obstacles, as the trajectory optimization stage has already generated an obstacle-avoidance path. Deviation along the direction of flight (tangential displacement) is relatively safer, primarily affecting the local speed and time distribution of the trajectory. Setting a larger semi-major axis α provides greater flexibility for the curvature adjustment and time reallocation. This allows the planner to adjust the spacing between control points to satisfy dynamic constraints (e.g., slowing down for a turn by moving control points closer) without being overly constrained by the initial path's exact length or speed profile.

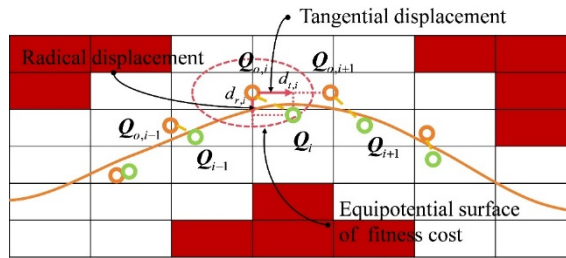


Fig. 4 Refined control points fit output control points of optimization stage while adjust dynamic feasibility (tangential and radical displacements are shown as the red arrows).

6. Global planning simulation

To identify the superiority of multi-stage trajectory planning method, the comparative studies with the state-of-the-art Trust-Region Filtered SCP (TRF-SCP) [15], GuSTO [14], and SFC-SCP [2] are conducted in the global planning scenario. A Motion-Primitive-based planner (RRT-MP) is also implemented using an RRT (Rapidly-exploring Random Tree) algorithm to search for a path and a predefined library of dynamically feasible motion primitives [24]

to connect the RRT nodes. In global planning experiments, the planner is required to generate a complete trajectory connecting the start and end points in once planning, rather than through multiple local replanning efforts. Since the source code of comparative methods is executed in MATLAB, the comparison experiments are run in MATLAB R2022b to ensure the fairness. The simulations are implemented on a desktop computer with Intel Core i7-13700KF 3.40 GHz CPU and 16 GB RAM.

6.1. Simulation scenario and parameter settings

The fixed-wing UAV is required to fly through multiple polyhedron obstacles randomly generated in a $200\text{ m} \times 200\text{ m} \times 100\text{ m}$ gridmap. The initial position, final position, initial heading angle, and the dynamics parameters of the fixed-wing UAV are shown in Table 1. We also provide the parameters of constraints on states and controls and the parameter settings of the proposed method.

Table 1 Simulation settings and parameters in global planning.

Parameter	Value
Initial position $p(t_0)$ (m)	(3, 3, 50)
Final position $p(t_f)$ (m)	(197, 197, 50)
Initial heading angle $\vartheta(t_0)$ ($^\circ$)	45
Mass of UAV m (kg)	12
Wing area S (m^2)	1.554
Drag coefficient at zero lift C_{D0}	0.072
Lift-induced drag coefficient k_0	0.033
Minimum speed $V_{\min}$ (m/s)	10
Maximum speed $V_{\max}$ (m/s)	14
Maximum thrust $T_{\max}$ (N)	160
Maximum roll angle $\phi_{\max}$ ($^\circ$)	40
Maximum load factor $n_{\max}$	2
Coefficient of smoothness cost μ_s	5
Coefficient of bound cost μ_b	10
Coefficient of collision cost μ_c	200
Coefficient of minimum curvature cost μ_r	30
Coefficient of fitness cost μ_f	30
Spheroidal metric α	5
Spheroidal metric $\tilde{n}$	1
Convergence tolerance of optimization ε_{opt}	0.001
Curvature limitation $\kappa_{\max}$	0.18
Number of control points $K+1$	40

6.2. Global planning results

Fig. 5 (a) displays the initial, optimized, and refined trajectories generated by the multi-stage planning method. The initial trajectory is generated by A* (red dash line). Then, the trajectory optimization is conducted to balance the smoothness, bound, and collision cost (green dot line). Last, the optimized trajectory is refined to satisfy the dynamical feasibility (magenta dash-dot line). The trajectory tracking result is shown in solid line.

Although the initial trajectory reaches the destination without violating any obstacle constraints, it does not obey dynamical constraints of fixed-wing UAVs. The following optimization stage strengthens the smoothness, limits the maximum velocity and acceleration, and further pushes the trajectory away from obstacles. In the trajectory refinement stage, the flight time and curvature are adjusted to satisfy the dynamical feasibility. The state variables and control

commands along the trajectory are shown in Fig. 5 (b). UAV flies through the obstacle environment with gentle state changes. Velocity $v(t)$, roll angle $\phi(t)$, heading angle $\vartheta(t)$, and thrust $T(t)$ all satisfy the state or control bound constraints. Table 2 shows that the time intervals are adjusted during the trajectory refinement process to meet the thrust constraints. Besides, the curvature is refined to adhere to the maximum curvature limitation of 0.18, in accordance with the dynamics of UAVs.

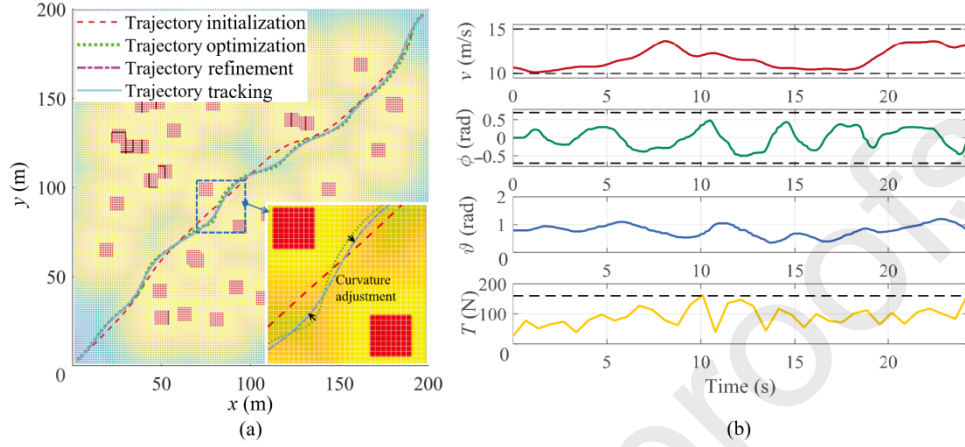


Fig. 5 Trajectory planning solution and corresponding states and controls: (a) Trajectory planning solution in environment with 36 polyhedron obstacles; (b) States and controls of generated trajectory.

Table 2 Intermediate data obtained from each part of trajectory plannings.

Stage of proposed method	Time interval (s)	Flight time (s)	Maximum curvature (m^{-1})
Trajectory initialization	0.579	23.16	N/A
Trajectory optimization	0.579	23.16	0.44
Trajectory refinement (Time reallocation)	0.635	25.40	0.26
Trajectory refinement (Curvature adjustment)	0.635	25.40	0.17

6.3. Planning performance comparison and analysis

To validate the performance of the proposed method, we test the proposed method on random cases with the number of obstacles varying from 10 to 40. For each case, we run 100 trials with random obstacle positions. The initial and final states are set to be the same as those in Section 6.2. For TRF-SCP [15], GuSTO [14], and SFC-SCP [2], the computation accuracy, i.e., convergence tolerance, is set to 10^{-3} , consistent with the proposed method. We compare the computation time, success rate, and flight time along the trajectories (optimality of trajectories) among these methods, and we analyze the trajectory tracking error of the proposed approach. Several trajectory planning results with random obstacles are shown in Fig. 6. The number of obstacles is set to 40, while the position of obstacles are generated randomly in Cases 1-3.

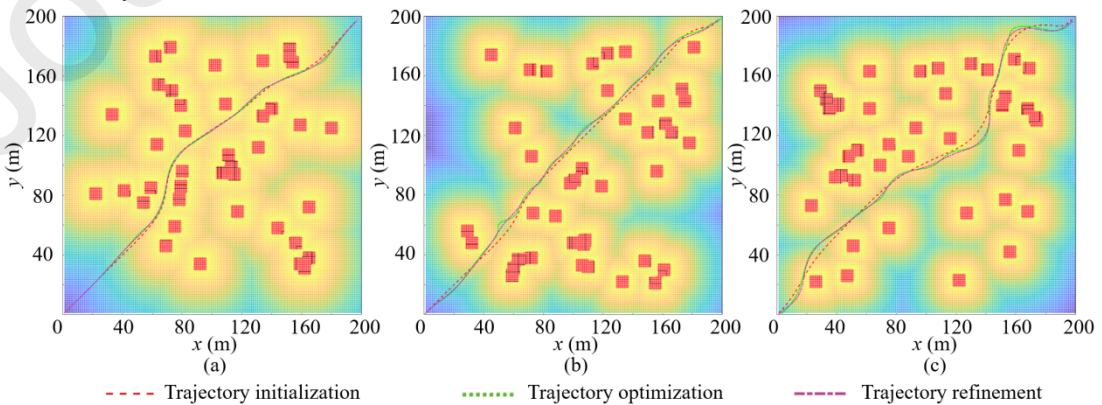


Fig. 6 Several trajectory planning results of proposed method in comparison test with random obstacles: (a) Case 1; (b) Case 2; (c) Case 3.

6.3.1. Computation time

In Table 3, the computation time on all cases is less than 300 ms, and the time assumption varies slightly as the number of obstacles increases. Since the trajectory optimization requires more iterations, this stage almost dominates the computationally expensive element of trajectory generation. The trajectory refinement stage ensures the dynamical feasibility, and only sacrifices a small amount of computation time (less than 55 ms).

Table 3 Computation time of global planning.

Number of obstacles	Mean / Max / Std (ms)		
	Initialization	Optimization	Refinement
10	11 / 54 / 0.004	194 / 200 / 0.003	35 / 44 / 0.027
20	13 / 56 / 0.005	193 / 199 / 0.002	33 / 45 / 0.025
30	16 / 57 / 0.005	194 / 199 / 0.002	34 / 44 / 0.026
40	19 / 64 / 0.005	195 / 210 / 0.002	34 / 53 / 0.026

Regarding computation time in Fig. 7, the proposed method demonstrates a significant advantage due to its lightweight planning framework and low-complexity hierarchical optimization problem. Based on differential flatness, we only need to plan in low-dimension space, and the complex spatiotemporal optimization is decoupled into temporal optimization and spatial optimization to further reduce the computation complex, which represents the main contribution of the proposed approach. SFC-SCP has higher efficiency compared to TRF-SCP, RRT-MP and GuSTO, since it customizes solver specifically for the trajectory planning problem.

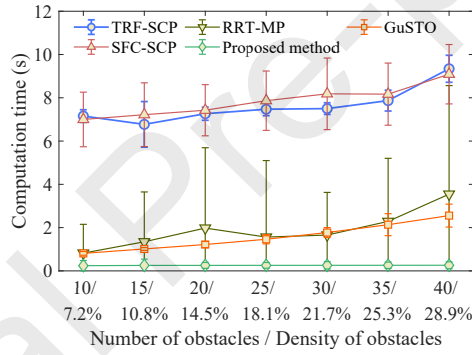


Fig. 7 Computation time comparison.

6.3.2. Success rate

As shown in Fig. 8, the success rate of the proposed method remains above 90% for all tests, and is significantly superior to that of the others when the obstacle density exceeds 14.5%. This advantage is attributed to the front-end path search, which provides an initial value for back-end optimization. In contrast, TRF-SCP and GuSTO rely solely on optimization, making them prone to getting trapped in local minima and yielding infeasible results, especially in environments with dense obstacles. For TRF-SCP, it may misjudge active constraints if the initial size of trust region is poorly set. Besides, the adaptive updating mechanism of TRF-SCP relies on dynamic constraint violations, but suffers from threshold tuning issues, i.e., high thresholds permit linearization errors and accuracy loss, and low ones cause frequent adjustments and convergence disruptions, especially in dense settings leading to infeasible subproblems. Additionally, straight-line initial guesses work well in sparse environments [15] but falter in complex ones, stranding solutions far from feasibility and preventing convergence of convex subproblems. SFC-SCP incorporates a front-end search, and it generates a Safety Flight Corridor (SFC) for back-end optimization. However, as the number of obstacles increases, conflicts arise between the SFC constraints and the dynamics of UAVs. The RRT-MP method achieves a comparable success rate with SFC-SCP in low and medium obstacle density environments. However, the success rate drops sharply with higher obstacle density (above 25.3%) for no existence of a feasible trajectory, or reaching a maximum RRT sampling limit, which is set to balance the computation time. For the planning failure of the proposed method, there exist several reasons. The time reallocation may fail to find a reasonable time scale, resulting in the excess of control limits. Furthermore, the curvature adjustment attempts to optimize control points regardless of obstacle-avoidance constraints. Although the tangential displacement is considered in this adjustment to ensure the safety, a collision with obstacles may occur when UAVs execute a sharp turning. Fortunately, UAV can replan its trajectory if the previous planning fails. Since the proposed method has short computation time and high success rate,

it can guide UAV fly in obstacle environments safely via frequent replanning, as shown in Section 7.

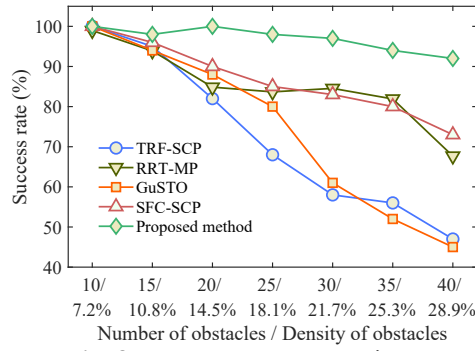


Fig. 8 Success rate comparison.

6.3.3. Optimality of trajectory results

In Fig. 9, the optimization objective of ours i.e., flight time, achieves comparable optimality to GuSTO and SFC-SCP, but it sacrifices some optimality compared with TRF-SCP since the three-stage framework. By spatiotemporal optimization decoupling, some optimality has been lost in the proposed method. However, the proposed method still shows great advantages over RRT-MP, as the solution space is heavily restricted by the discrete, pre-defined set of motion primitives, limiting the UAV's ability to find a more efficient route. In summary, the proposed approach offers notable stability and efficiency benefits, sacrificing a small degree of optimality.

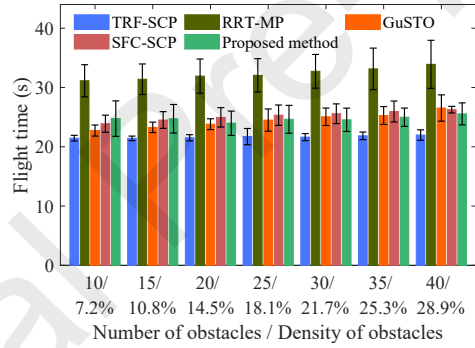


Fig. 9 Trajectory result optimality comparison.

6.3.4. Trajectory tracking error

The dynamically feasible and safe trajectory generated by the proposed method is executed using the MATLAB UAV Toolbox [33] with fixed-wing UAV dynamics. A PD autopilot controller is utilized to achieve the trajectory tracking under coordinated-turn and zero sideslip flight conditions. We record the maximum trajectory tracking error along the whole trajectory in each trial, and the statistics are provided in Table 4. The mean tracking error grows slightly as the number of obstacles increases, and always keeps below 2.2 m. In higher density environments, the method generates more detour trajectory, resulting in larger tracking error. The Maximum tracking error (Max) is due to that the random obstacles may locate near the initial position and UAV needs an aggressive trajectory to avoid the high-risk obstacles. The Standard deviation (Std) indicates that the tracking error throughout the process is relatively stable.

Table 4 Trajectory tracking error.

Number of obstacles / Density of obstacles	Mean (m)	Max (m)	Std (m)
10 / 7.2%	1.01	3.85	0.41
15 / 10.8%	1.78	4.42	0.55
20 / 14.5%	2.07	6.37	0.99
25 / 18.1%	1.99	5.52	1.01
30 / 21.7%	1.93	6.58	0.85

35 / 25.3%	2.18	6.59	1.18
40 / 28.9%	2.08	6.88	1.19

6.4. Parameter sensitivity analysis

The selection of coefficient in Eq.(4) is a hyperparameter tuning problem, and the selection has impact to the planning results. Therefore, we set key parameters according to the empirical selection in Ref. 32. The selection of μ_s , μ_b , μ_c need to ensure the obstacle avoidance primarily. It is considered that dynamics cost J_b takes precedence over smoothness cost J_s . Therefore, μ_s , μ_b , μ_c are set to 5, 10, 200, respectively.

In addition to the empirical selection, we conduct sensitivity analysis to explore the performance of this proposed method. In the sensitivity analysis, μ_s and μ_b keep unchanged, collision cost μ_c is increased gradually. The results are shown in Fig. 10. With relatively small μ_c , the optimization problem tends to find smoother trajectory, and the trajectory is close to the obstacle. As μ_c increases, the smoothness is reduced, but the trajectory stays farther away from the obstacles, which make the UAVs safer.

However, if we set unreasonable parameter with very large μ_c , it may render the problem infeasible by imposing overly stringent penalties on collision risks, often clashing with UAV dynamics constraints. Specifically, μ_c is increased from 2000 to 10000 while keeping $\mu_s = 5, \mu_b = 10$ constant. We conduct tests as shown in Fig. 11, and the trajectory cannot satisfy the dynamics constraints with $\mu_c = 5000$. According to the sensitivity analysis, we suggest that the coefficients μ_s and μ_b can be set on the same scale and the coefficient of collision cost μ_c is one order of magnitude higher than μ_s and μ_b to ensure the obstacle avoidance and dynamical feasibility, i.e., a suitable and conservative range for the collision cost coefficient as [200, 2000]. In addition, we find that this parameter has slight impact on the computation time if there exists a solution, since the complex of the trajectory optimization problem remain unchanged.

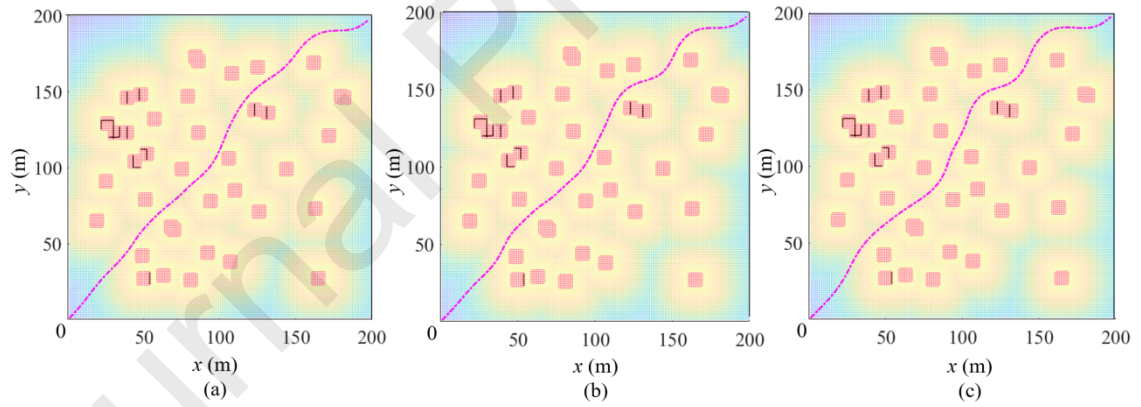


Fig. 10 Parameter sensitivity analysis of cost coefficients in Eq. (4): (a) $\mu_s = 5, \mu_b = 10, \mu_c = 20$; (b) $\mu_s = 5, \mu_b = 10, \mu_c = 200$; (c) $\mu_s = 5, \mu_b = 10, \mu_c = 2000$.

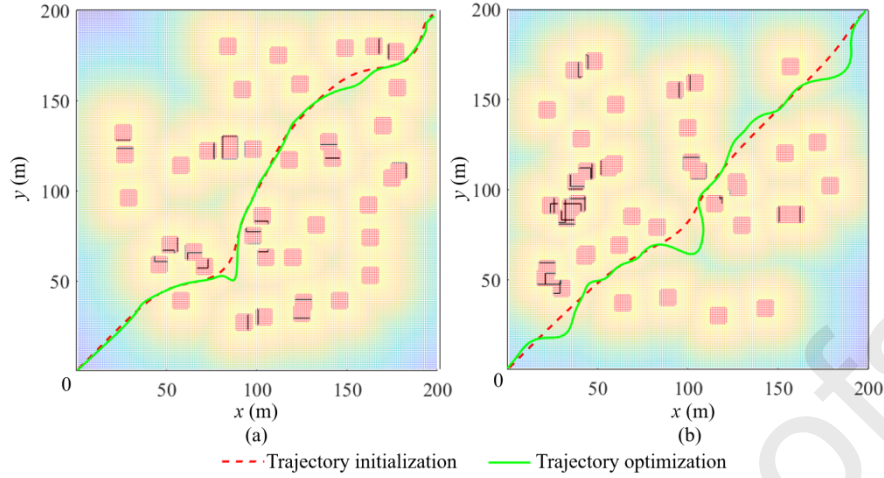


Fig. 11 Trajectory planning results in dense obstacle environments: (a) $\mu_s = 5, \mu_b = 10, \mu_c = 5000$; (b) $\mu_s = 5, \mu_b = 10, \mu_c = 10000$.

7. Local replanning simulation

We deploy the proposed method into local replanning framework, and the corresponding algorithm is rewritten in C++ and implemented on ROS Noetic. We also establish the trajectory tracking and dynamics integration module for fixed-wing UAVs to realize simulated flight. The replanning simulation experiments are implemented on a desktop with the same hardware as Section 6.

For the conditions of replanning, we combine a time-triggered mechanism with an event-triggered mechanism. The planner will perform replanning every 0.1 seconds. If the generated trajectory collides with an obstacle or if the previous trajectory planning fails, it will trigger a new replanning.

7.1. Simulation scenario of local replanning

The map size is 600 m×100 m. Numbers of obstacles are randomly placed in the map. According to the receding horizon strategy, the proposed planner generates local trajectory and utilizes a local map within the limited sensing range of UAVs. The initial and final positions, sensing range, sensing radius, and the sensing rate are listed in Table 5. The algorithm parameter settings are set same with that in Section 6.1.

Table 5 Simulation settings and parameters in local replanning.

Parameter	Value
Initial position $p(t_0)$ (m)	(-250, 0, 16)
Final position $p(t_f)$ (m)	(250, 0, 15)
Sensing range S_r (°)	180
Sensing radius R_s (m)	30
Sensing frequency S_f (Hz)	20

The generated local trajectory is executed by the tracking controller with double-closed-loop, and the sketch of the controller is illustrated in Fig. 12. The position loop controls course angle χ , roll angle ϕ , height h , and air speed V_a at a rate of 20 Hz. The attitude loop controls angle of aileron δ_a , angle of elevator δ_e , angle of rudder δ_r , and throttle u_t at 100 Hz. The controller is formulated with separate lateral and longitudinal control accurate to rudder surfaces, providing high fidelity tracking simulation for planning algorithm validation. Specifically, the lateral and longitudinal controllers are decoupled and designed in a double-closed loop structure. For the lateral controller, the inner loop contains a PD controller to achieve the roll attitude hold, while the outer loop contains a PI controller to conduct course hold with the control of the ailerons. The yaw damper is tuned to restrain sideslip to zero error by adjusting the gain of rudder angle. For the longitudinal controller, a PD controller is deployed in the inner loop to achieve pitch attitude hold, while a PI controller is deployed to hold flight altitude with the control of the elevators.

The air speed is held by a PI controller with the control of the throttle. All controllers are tuned manually with the lateral-longitudinal and inner-outer order. Using this replanning framework, the proposed method enables flexible reaction to newly perceived obstacles. Once new obstacles are perceived or the current trajectory is timeout, the method will replan a local trajectory.

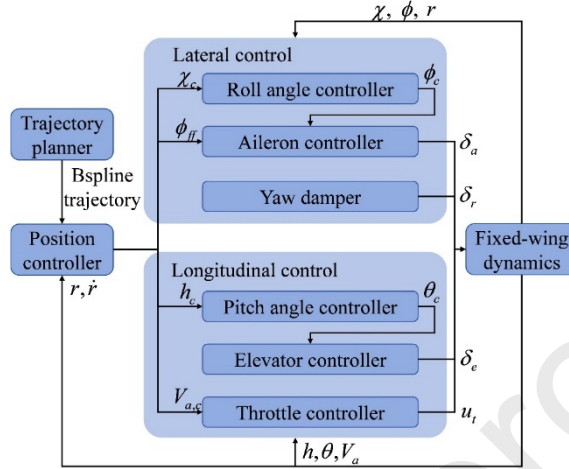


Fig. 12 Sketch of fixed-wing UAV lateral and longitudinal controller implementation in local replanning simulation.

forward control value, respectively.

7.2. Local replanning in random polyhedron environment

Fig. 13 shows the overall local replanning results in a random case with multiple polyhedron obstacles. In Fig. 13 (a), the narrow obstacle-free space induces challenge to trajectory planning subject to the limited flight performance of fixed-wing UAVs. From the results, it can be found that the generated local trajectory can guide UAV fly through the obstacle environment. Figs. 13 (b)-(e) show that the UAV can track the trajectory with coordinated roll angle at the velocity over 10 m/s, which is attributed to the time reallocation and curvature adjustment. Fig. 14 shows the multi-stage planning process, and UAV executes the refined trajectory.

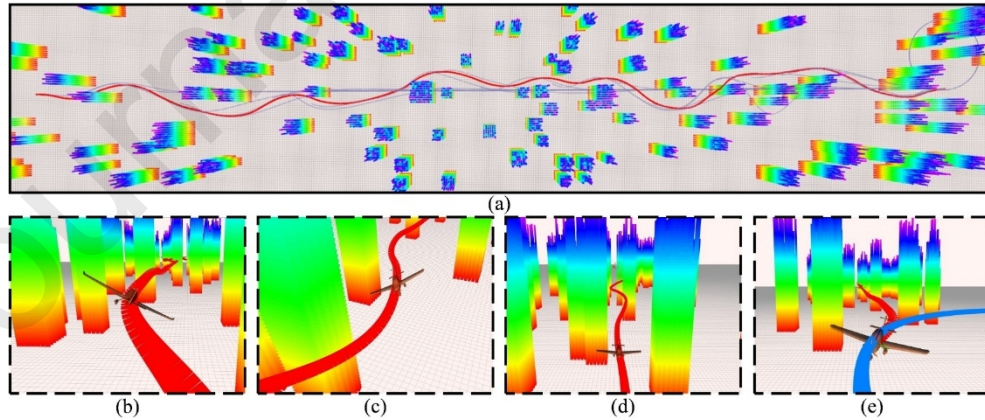


Fig. 13 A fixed-wing UAV traverses an unknown environment with polyhedron obstacles from the left side to the right side: (a) Overall top-view flight trajectory with 52.0 s total flight time and 531.1 m flight distance; (b)-(e) Flight attitude of fixed-wing UAV at time of 13 s, 26 s, 39 s, and 52 s, respectively. The blue circle represents the hovering trajectory. UAV will switch to hovering state when arriving at final position.

The computation time of local replanning is provided in Table 6. We perform three tests with random obstacles, and the maximum computation time of each stage in the replanning process is recorded for each case. It can be seen that the total computation time is below 20 ms. The initialization costs most of the runtime, since this stage includes A*-based path search and ESDF map establishment. In the local replanning, the optimization stage only uses few control

points (less than 10), thereby a low-dimensional optimization problem is constructed and the corresponding solution time is extremely short (around 1 ms). Considering that the maximum flight velocity and CPU-time of replanning are 15 m/s and 20 ms, the maximum flight distance during the replanning process is less than 0.3 m. Therefore, UAV has adequate time and space to avoid new obstacles within the 30 m sensing range. The number of local re-planning times is influenced by the distance between the start and end points, as well as the re-planning horizon. A longer distance necessitates more local re-planning times. All in all, the replanning results and the success rate analysis in Section 6.3 indicate the high robustness merit of the proposed trajectory planner.

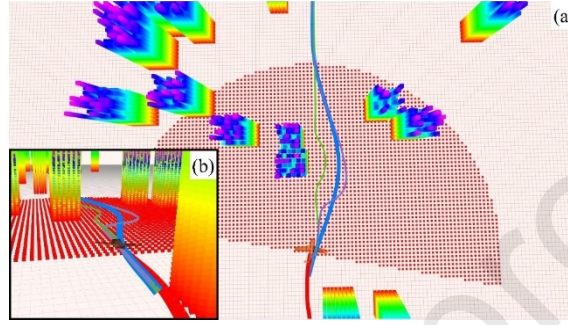


Fig. 14 Demonstration of multi-stage trajectory planning in simulations: (a) Top view; (b) Side view. Red sector area indicates the sensing range. The green, purple, and blue curves denote the trajectories generated in initialization (1st stage), optimization (2nd stage), and refinement (3rd stage), respectively. The red curve represents the trajectory tracking results of the fixed-wing UAV. As showed in the side view, UAV tracks the blue refined trajectory.

Table 6 Computation time of local replanning.

Random case	Replanning times	Maximum computation time (ms)		
		Initialization	Optimization	Refinement
1	29	9.868	0.682	0.781
2	26	8.553	1.059	0.596
3	27	10.174	1.063	0.825

7.3. Local replanning in environment with irregular obstacles

To further validate the planner's generality and robustness, we conduct experiments in a challenging environment populated with highly irregular and concave obstacles. Such non-convex geometries introduce a significant challenge for gradient-based optimization methods, as they can create numerous local minima that may trap a planner. The proposed hierarchical and multi-stage approach is specifically designed to mitigate this risk by using a robust front-end search to provide a high-quality initial guess for the back-end optimization.

The simulation setup and results are shown in Fig. 15. The environment consists of a narrow and winding corridor populated with C-shaped obstacles. Fig. 15 (a) shows the final executed trajectory (red line), demonstrating that the UAV successfully navigates the entire cluttered environment. The sequence in Figs. 15 (b)-(d) highlights a critical replanning event that demonstrates the effectiveness of the proposed method. In Fig. 15 (b), a distant C-shaped obstacle lies outside the UAV's 30 m sensing range, and the currently planned trajectory (blue curve) passes through this obstacle. This creates a potential local minimum trap for the subsequent planning cycle. As the UAV moves forward, the non-convex obstacle enters the perception range, triggering a replan, as shown in Fig. 15 (c). Instead of attempting a minor and gradient-based modification that would likely get caught in the obstacle's concave trap, the A*-based initialization stage generates a new trajectory that intelligently navigates all the way around the obstacle. The subsequent optimization and refinement stages then smooth this safe initial path (green and blue curves) into a dynamically feasible trajectory. Fig. 15 (d) shows the successful outcome.

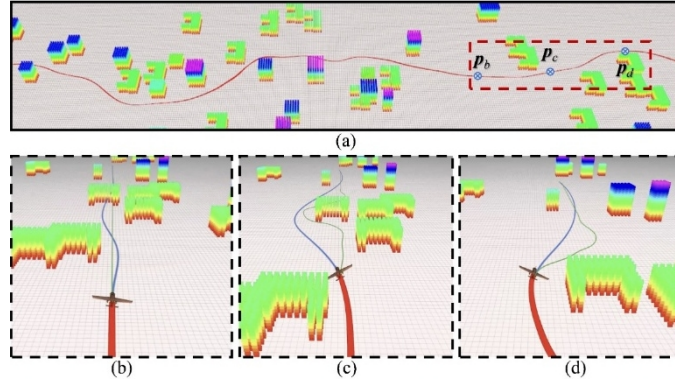


Fig. 15 Local replanning in a cluttered environment with irregular, C-shaped obstacles: (a) Top-down view of complete, successful trajectory; (b) Planner generates a trajectory before a distant non-convex obstacle is within sensing range; (c) Upon detecting obstacles, a replan is triggered; A*-based initialization provides a safe path that bypasses obstacle's concave trap; (d) UAV successfully executes new trajectory.

7.4. Local replanning in dynamic environment

To evaluate the planner's performance in a dynamic environment, we design a simulation scenario with moving obstacles, as depicted in Fig. 16 and Fig. 17. The environment contains a mix of static prisms and dynamic cylindrical obstacles. The cylinders move back and forth along randomized line segments (0-10 m length) or sinusoidal curves (0-10 m length and 0-8 m amplitude) at random speeds (0-4 m/s).

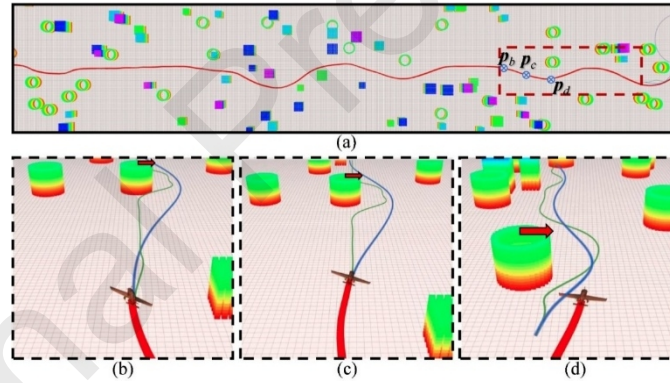


Fig. 16 Local replanning in a dynamic environment with moving obstacles moving along straight lines: (a) Top-down view of complete, successful trajectory among static prisms and dynamic cylinders; (b)-(d) UAV executes planned trajectory, successfully avoiding moving obstacle.

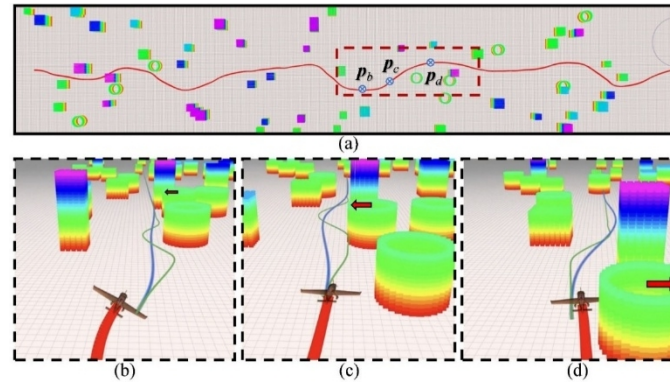


Fig. 17 Local replanning in a dynamic environment with moving obstacles moving along sinusoidal curves: (a) Top-down view of the complete, successful trajectory among static prisms and irregular dynamic cylinders; (b)-(d) UAV

executes planned trajectory, successfully avoiding moving obstacle.

The proposed planner demonstrates generalization to dynamic environments due to its efficient replanning capability. It takes less than 20 ms in total per replanning cycle. The subplots in Fig. 16 and Fig. 17 illustrate a typical avoidance maneuver. For cylindrical obstacles moving in a straight line, Fig. 16 (a) shows the complete executed trajectory (red line) successfully navigating through the entire environment. The sequence begins in Fig. 16 (b), where a trajectory is formed based on the current obstacle positions. As the UAV and obstacles move, a replan is triggered. In Fig. 16 (c), the A*-based initialization stage (green curve) generates a new collision-free path that accounts for the moving obstacle's updated position. Even as the cylinder continues to move toward the UAV's flight path, as seen in Fig. 16 (d), the planner's high replanning frequency allows it to continuously update the trajectory and ensure safety. While Fig. 17 shows a dangerously close interaction with the irregular dynamic obstacle. Because our current method is purely reactive to dynamic environment and does not include a speed prediction model for the obstacle, the planned trajectory (blue curve) is generated based only on the obstacle's current position. As the obstacle moves along its sinusoidal path (indicated by the red arrow), the planner is forced to react. This results in a trajectory that bulges towards the obstacle's future direction of travel, causing the UAV to pass dangerously close, as seen in Fig. 17 (d). Note that our current approach is demonstrated for low-speed dynamic obstacles. To achieve more aggressive flight in highly dynamic environments, the planner would need to be augmented with more advanced techniques, i.e. obstacle method for modeling dynamic environments.

7.5. Local replanning under wind disturbances

In practical environments, fixed-wing UAVs are indeed sensitive to effects like wind, and it is crucial to analyze the tracking accuracy and robustness under disturbances [34]. To empirically evaluate the method under disturbances, we conducted simulation experiments to analyze the system's tracking performance under various static wind conditions. As shown in Fig. 18 (a), a reference trajectory is planned. The UAV tracks this path in static wind fields ranging from $v_w = 0$ m/s to $v_w = 5$ m/s, with the wind direction indicated in Fig. 18 (b), which shows the trajectory executed result. A quantitative analysis of the cross-tracking error is presented in the box plot Fig. 19. The results indicate that the tracking performance remains robust in wind speeds from 0 m/s to 4 m/s, with the Root Mean Square Error (RMSE) staying below 1.15 meters. At 5 m/s, a significant increase in tracking error is observed (RMSE of 3.43 m and maxima of 12.72 m), and simulations at 6 m/s resulted in tracking failure. This analysis confirms that the proposed method can effectively manage moderate wind disturbances implicitly.

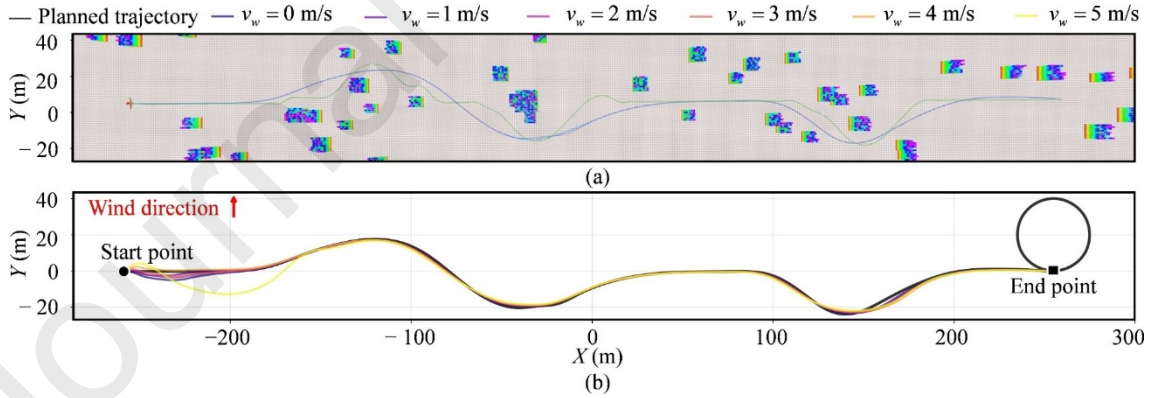


Fig. 18 Local replanning under wind disturbances. (a) Planned reference trajectory by proposed method; (b) Trajectory tracking results under different wind disturbances.

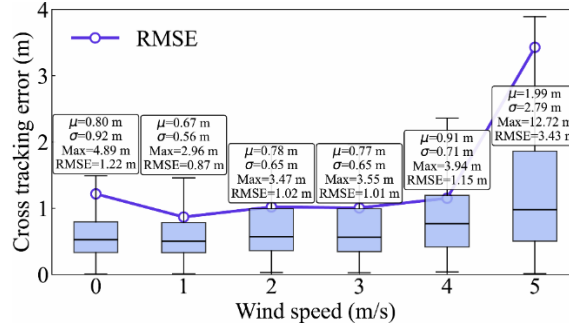


Fig. 19 Cross-tracking error under wind disturbances.

7.6. Comparison with a tightly-coupled planning architecture

The proposed multi-stage pipeline is architected as a sequential and loosely-coupled framework, separating global pathfinding from local refinement. To evaluate the trade-offs of this architectural choice, we investigate a more tightly-coupled alternative that incorporates kinematic feasibility constraints directly into the initial search stage.

For this comparison, we develop a hybrid A* algorithm tailored for the fixed-wing UAV model. Unlike the proposed modified A* which searches in the 3D position space, the hybrid A* expands nodes in a 4D state space, including position and heading angle. It generates successor states using kinematically feasible motion primitives based on Dubins curves, inherently respecting the minimum turning radius. For the tests, we use a heading angle resolution of 5° , a map resolution of 2.5 m, and a maximum steering angle of 0.2 rad, corresponding to a minimum turning radius of 12.5 m. At each node, 7 discrete steering angles from -0.2 to 0.2 rad are used to generate motion primitives.

We then conduct a comparative study, with the results shown in Fig. 20. The comparison reveals critical trade-offs in path quality and performance.

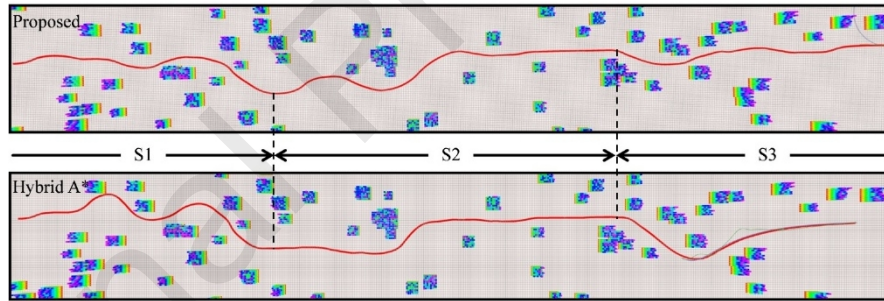


Fig. 20 Comparison of trajectories generated by proposed planner and tightly-coupled planner utilizing hybrid A* with Dubins curve motion primitives in same scenario.

Trajectory quality: In segment S1, the hybrid A* path is forced to make a wide, conservative counter-maneuver to ensure its curvature limit is respected during the search. In contrast, the proposed method's path is shorter and more direct. In segment S2, the hybrid A* path is notably smoother due to a yaw penalty in its heuristic. However, in segment S3, the constrained search space of hybrid A* forced it to abandon the shortest path through a gap between two obstacles, opting for a longer route that the proposed method avoided.

Computational performance: The most decisive factor was computational performance. The hybrid A* search was substantially slower due to the expansion from a 3D to a 4D search space. In a representative flight, the hybrid A*-guided planner required 22 replans with an average search time of 111.78 ms per replan. This severely compromises real-time capability compared to the 20 ms total replanning time of the proposed method.

While a tightly integrated search like hybrid A* can enforce kinematic feasibility from the start, the experiments showed that this comes at a significant cost to computational speed and can lead to overly conservative or sub-optimal path choices. The proposed loosely-coupled architecture has provided a better balance of trajectory quality, planning robustness, and, most critically, real-time performance.

8. Outdoor flight validation

To validate the executability of the generated trajectories, we conducted outdoor flight experiments using a fixed-

wing UAV equipped with a PX4 flight controller. The UAV's state estimation is achieved through integrated GPS and IMU measurements, providing real-time position, velocity, and attitude data.

The obstacle-avoid trajectory is generated and is uploaded to the PX4 autopilot before takeoff. The UAV is manually hand-launched and stabilized in loiter mode. Upon achieving stable flight, the system transitioned to trajectory tracking mode, autonomously guiding the UAV to track the planned trajectory. After executing the trajectory, the UAV enters an autonomous landing sequence to terminate the test.

As illustrated in Fig. 21 (a), the actual flight trajectories closely align with the generated trajectories, demonstrating effective tracking performance. The lateral tracking errors remained around 5 m, within the acceptable range considering typical GPS positioning uncertainties (2-3 m). Figs. 21 (b)-(d) show the flight process of trajectory tracking, landing, and takeoff, respectively. Fig. 22 provides the detail flight data including altitude, velocity, and attitude. The velocity is maintained at ≤ 15 m/s throughout the test, consistent with predefined safety constraints. The generated trajectories enable successful flight while respecting minimum turn radius limitations. These experiment results confirm the practical trackability of the generated trajectories under real-world operational conditions. The consistency between planned and executed paths, coupled with compliant velocity profiles, validates both the feasibility and dynamic compatibility of the proposed planning framework.

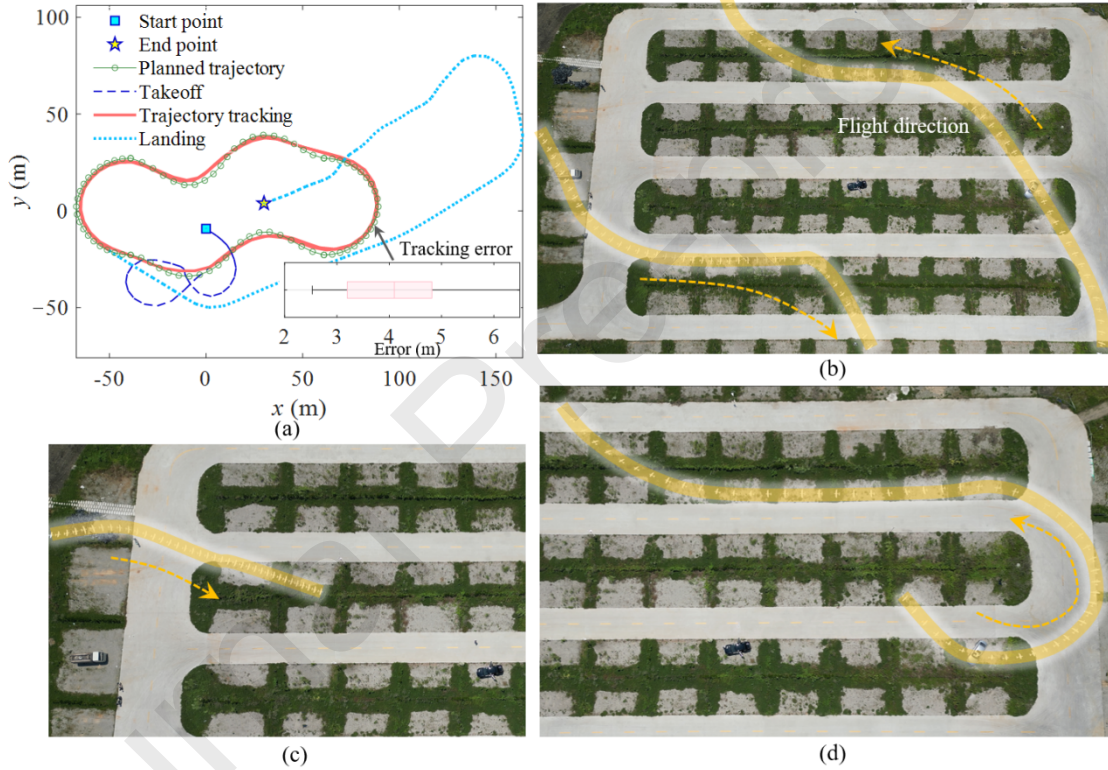


Fig. 21 Illustration of outdoor flight experiments: (a) Overall planned trajectory, tracking trajectory, and tracking error; (b) Flight states during trajectory tracking; (c) Flight states during landing; (d) Takeoff and transition to trajectory tracking mode.

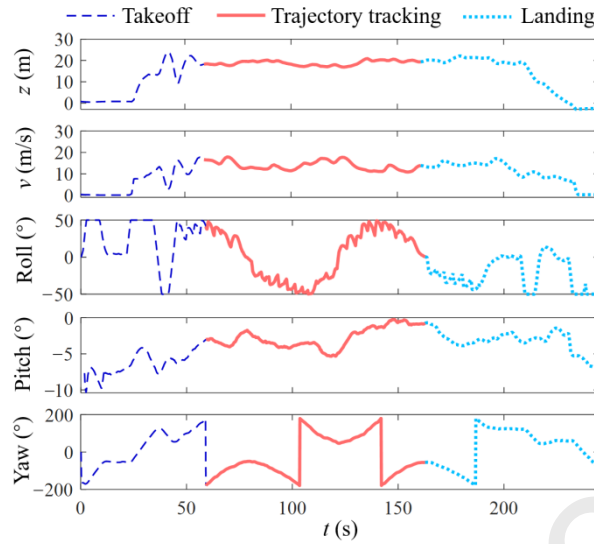


Fig. 22 Demonstration of flight states in outdoor flight experiment.

9. Conclusions

The multi-stage trajectory planning method is proposed to generate feasible and safe trajectory for fixed-wing UAVs efficiently and robustly. This method leverages the advantages of graph search algorithm and gradient descent optimization. Besides, the trajectory refinement is designed by utilizing differential flatness to ensure the dynamical feasibility. Simulation experiments on global planning scenarios show that the proposed method outperforms the comparative optimization method in terms of computation efficiency and success rate. The local replanning tests demonstrate that the proposed method can generate obstacle-free trajectory within sensing range costing less than 20 ms. The simulation results also validate that the generated trajectory can be tracked accurately by using standard tracking controller. In future work, we will consider factors such as wind interference in trajectory planning problems and develop learning-integrated trajectory planning method to further enhance the algorithm's computational efficiency and robustness.

Acknowledgements

This work is supported in part by the Huzhou Natural Science Foundation, China (No. 2024YZ04), the Tsinghua-Toyota Joint Research Fund, China, the National Natural Science Foundation of China (Nos. 62273195 and 62203256), and the Beijing Natural Science Foundation, China (No. L252095).

References

- [1] Chung SJ, Paranjape AA, Dames P, et al. A survey on aerial swarm robotics. *IEEE Trans Robot* 2018;34(4):837–55.
- [2] Sun J, Xu GT, Wang Z, et al. Safe flight corridor constrained sequential convex programming for efficient trajectory generation of fixed-wing UAVs. *Chin J Aeronaut* 2025;38(1):103174.
- [3] Persson SM, Sharf I. Sampling-based A* algorithm for robot path-planning. *Int J Robot Res* 2014;33(13):1683–708.
- [4] Kamyar R, Taheri E. Aircraft optimal terrain/threat-based trajectory planning and control. *J Guid Contr Dyn* 2014;37(2):466–83.
- [5] Foehn P, Romero A, Scaramuzza D. Time-optimal planning for quadrotor waypoint flight. *Sci Robot* 2021;6(56):eabh1221.
- [6] Betts JT. Survey of numerical methods for trajectory optimization. *J Guid Contr Dyn* 1998;21(2):193–207.
7. Patterson MA, Rao AV. GPOPS-II: A Matlab software for solving multiple-phase optimal control problems using hp-adaptive gaussian quadrature collocation methods and sparse nonlinear programming. *ACM Trans Math Softw* 2014;41(1):1–37.

- [8] Chai RQ, Tsourdos A, Savvaris A, et al. Real-time reentry trajectory planning of hypersonic vehicles: A two-step strategy incorporating fuzzy multiobjective transcription and deep neural network. *IEEE Trans Ind Electron* 2020;67(8):6904–15.
- [9] Wu JF, Wang HL, Liu YH, et al. Learning-based fixed-wing UAV reactive maneuver control for obstacle avoidance. *Aerosp Sci Technol* 2022;126:107623.
- [10] Yan C, Wang C, Xiang XJ, et al. Collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments: A task-specific curriculum- based MADRL approach. *IEEE Trans Neural Netw Learn Syst* 2023;35(8):10894–908.
- [11] Luis CE, Schoellig AP. Trajectory generation for multiagent point-to-point transitions via distributed model predictive control. *IEEE Robot Autom Lett* 2019;4(2):375–82.
- [12] Long T, Cao Y, Xu GT, et al. Real-time multi-quadrotor trajectory generation via distributed receding architecture and hierarchical planning in complex environments. *ISA Trans* 2023;136:715–26.
- [13] Wang Z, Liu L, Long T. Minimum-time trajectory planning for multi-unmanned-aerial-vehicle cooperation using sequential convex programming. *J Guid Contr Dyn* 2017;40(11):2976–82.
- [14] Bonalli R, Cauligi A, Byland A, et al. GuSTO: Guaranteed sequential trajectory optimization via sequential convex programming. 2019 international conference on robotics and automation (ICRA). Piscataway: IEEE Press; 2019.
- [15] Xu GT, Long T, Wang Z, et al. Trust-region filtered sequential convex programming for multi-UAV trajectory planning and collision avoidance. *ISA Trans* 2022;128:664–76.
- [16] Chai RQ, Guo YL, Zuo ZY, et al. Cooperative motion planning and control for aerial-ground autonomous systems: Methods and applications. *Prog Aerosp Sci* 2024;146:101005.
- [17] Zhou BY, Gao F, Wang LQ, et al. Robust and efficient quadrotor trajectory generation for fast autonomous flight. *IEEE Robot Autom Lett* 2019;4(4):3529–36.
- [18] Liu SK, Atanasov N, Mohta K, et al. Search-based motion planning for quadrotors using linear quadratic minimum time control. 2017 IEEE/RSJ international conference on intelligent robots and systems (IROS). Piscataway: IEEE Press; 2017.
- [19] Ding WC, Gao WL, Wang KX, et al. An efficient B-spline-based kinodynamic replanning framework for quadrotors. *IEEE Trans Robot* 2019;35(6):1287–306.
- [20] Tordesillas J, Lopez BT, Everett M, et al. FASTER: Fast and safe trajectory planner for navigation in unknown environments. *IEEE Trans Robot* 2021;38(2):922–38.
- [21] Li J, Long T, Sun J, et al. Differential flatness-based spatial-temporal trajectory optimization for fixed-wing UAVs in urban environments. *Acta Aeronautica et Astronautica Sinica* 2025;46(9):531369 [Chinese].
- [22] Wang ZP, Zhou X, Xu C, et al. Geometrically constrained trajectory optimization for multicopters. *IEEE Trans Robot* 2022;38(5):3259–78.
- [23] Chitsaz H, LaValle SM. Time-optimal paths for a dubins airplane. 2007 46th IEEE conference on decision and control. Piscataway: IEEE Press; 2007.
- [24] Paranjape AA, Meier KC, Shi XC, et al. Motion primitives and 3D path planning for fast flight through a forest. *Int J Robot Res* 2015;34(3):357–77.
- [25] Sun SH, Romero A, Foehn P, et al. A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. *IEEE Trans Robot* 2022;38(6):3357–73.
- [26] Mellinger D, Kumar V. Minimum snap trajectory generation and control for quadrotors. 2011 IEEE international conference on robotics and automation. Piscataway: IEEE Press; 2011.
- [27] Han ZC, Wu YW, Li T, et al. An efficient spatial-temporal trajectory planner for autonomous vehicles in unstructured environments. *IEEE Trans Intell Transp Syst* 2024;25(2):1797–814.
- [28] Bry A, Richter C, Bachrach A, et al. Aggressive flight of fixed-wing and quadrotor aircraft in dense indoor environments. *Int J Robot Res* 2015;34(7):969–1002.
- [29] Dai R, Cochran JE Jr. Three-dimensional trajectory optimization in constrained airspace. *J Aircr* 2009;46(2):627–34.
- [30] Alturbeh H, Whidborne JF. Real-time obstacle collision avoidance for fixed wing aircraft using B-splines. 2014 UKACC international conference on control (CONTROL). Piscataway: IEEE Press; 2014.
- [31] Usenko V, von Stumberg L, Pangercic A, et al. Real-time trajectory replanning for MAVs using uniform B-splines and a 3D circular buffer. 2017 IEEE/RSJ international conference on intelligent robots and systems (IROS). Piscataway: IEEE Press; 2017.
- [32] Zhou X, Wang ZP, Ye HK, et al. EGO-planner: An ESDF-free gradient-based local planner for quadrotors. *IEEE Robot Autom Lett* 2021;6(2):478–85.
- [33] MathWorks. UAV Toolbox [Internet]. MATLAB R2022b [updated 2022 Aug 24; cited 2025 Dec 25]. Available from: <https://www.mathworks.com/help/uav>.
- [34] Chai RQ, Tsourdos A, Gao HJ, et al. Dual-loop tube-based robust model predictive attitude tracking control for spacecraft with system constraints and additive disturbances. *IEEE Trans Ind Electron* 2021;69(4):4022–33.

Journal Pre-proofs